🐍

150+ Basic Python Programs

🐍 150+ Basic Python Programs

> 💡 **How to use this:** Every program is self-contained. Copy, run, understand. Programs go from dead simple to genuinely useful. Master these and you have a complete Python foundation.

🔵 1. Fundamentals (01–15)

> 🎯 The absolute starting point. Every Python programmer writes these first.

01 — Hello World

```
print("Hello, World!")
```

02 — Variables and Print

```
name = "Alice"
age = 25
print(f"Name: {name}, Age: {age}")
```

03 — User Input

```
name = input("Enter your name: ")
print(f"Hello, {name}!")
```

04 — Swap Two Variables

```
a, b = 10, 20
a, b = b, a
print(f"a={a}, b={b}")
```

05 — Check Variable Type

```python
x = 3.14
print(type(x))          # <class 'float'>
print(isinstance(x, float))  # True
```

06 — Multiple Assignment

```python
x = y = z = 100
a, b, c = 1, 2, 3
print(x, y, z, a, b, c)
```

07 — Basic Arithmetic

```python
a, b = 15, 4
print(a + b)   # 19  addition
print(a - b)   # 11  subtraction
print(a * b)   # 60  multiplication
print(a / b)   # 3.75 division
print(a // b)  # 3   floor division
print(a % b)   # 3   modulus
print(a ** b)  # 50625 power
```

08 — Convert Celsius to Fahrenheit

```python
celsius = float(input("Celsius: "))
fahrenheit = (celsius * 9/5) + 32
print(f"{celsius}°C = {fahrenheit}°F")
```

09 — Area of a Circle

import math

```python
import math
r = float(input("Radius: "))
area = math.pi * r ** 2
print(f"Area = {area:.2f}")
```

10 — Simple Interest Calculator

```python
p = float(input("Principal: "))
r = float(input("Rate %: "))
t = float(input("Time (years): "))
si = (p * r * t) / 100
print(f"Simple Interest = {si:.2f}")
```

11 — BMI Calculator

```python
weight = float(input("Weight (kg): "))
height = float(input("Height (m): "))
bmi = weight / height ** 2
print(f"BMI = {bmi:.2f}")
```

12 — Odd or Even

```python
n = int(input("Enter number: "))
print("Even" if n % 2 == 0 else "Odd")
```

13 — Positive, Negative or Zero

```python
n = float(input("Enter number: "))
if n > 0:   print("Positive")
elif n < 0: print("Negative")
else:       print("Zero")
```

14 — Largest of Three Numbers

```python
a, b, c = 10, 25, 18
largest = max(a, b, c)
print(f"Largest: {largest}")
```

15 — Leap Year Check

```python
year = int(input("Year: "))
if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
    print("Leap Year")
```

```
else:
    print("Not a Leap Year")
```

🟣 2. Math & Numbers (16–30)

> 🎯 Number manipulation, arithmetic logic, and math functions used in real programs.

16 — Factorial (loop)

```
n = int(input("n: "))
result = 1
for i in range(1, n + 1):
    result *= i
print(f"{n}! = {result}")
```

17 — Factorial (recursion)

```
def factorial(n):
    return 1 if n <= 1 else n * factorial(n - 1)
print(factorial(6))  # 720
```

18 — Fibonacci Series

```
n = 10
a, b = 0, 1
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

19 — Check Prime Number

```
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0: return False
```

```
    return True
print(is_prime(17))  # True
```

20 — Prime Numbers in a Range

```
primes = [n for n in range(2, 50) if all(n % i != 0 for i i
n range(2, int(n**0.5)+1))]
print(primes)
```

21 — GCD of Two Numbers

```
import math
a, b = 48, 18
print(f"GCD = {math.gcd(a, b)}")  # 6
```

22 — LCM of Two Numbers

```
import math
a, b = 4, 6
print(f"LCM = {math.lcm(a, b)}")  # 12
```

23 — Sum of Digits

```
n = 12345
digit_sum = sum(int(d) for d in str(n))
print(f"Sum of digits: {digit_sum}")  # 15
```

24 — Reverse a Number

```
n = 12345
reversed_n = int(str(n)[::-1])
print(reversed_n)  # 54321
```

25 — Palindrome Number Check

```
n = 121
print("Palindrome" if str(n) == str(n)[::-1] else "Not Pali
ndrome")
```

26 — Armstrong Number Check

```
n = 153
digits = len(str(n))
print("Armstrong" if sum(int(d)**digits for d in str(n)) ==
n else "Not Armstrong")
```

27 — Perfect Number Check

```
n = 28
divisor_sum = sum(i for i in range(1, n) if n % i == 0)
print("Perfect" if divisor_sum == n else "Not Perfect")  #
Perfect
```

28 — Power Without Built-in

```
def power(base, exp):
    result = 1
    for _ in range(exp):
        result *= base
    return result
print(power(2, 10))  # 1024
```

29 — Square Root Without math

```
n = 144
root = n ** 0.5
print(f"√{n} = {root}")  # 12.0
```

30 — Count Digits in a Number

```
n = 987654321
print(f"Digits: {len(str(abs(n)))}")  # 9
```

🟠 3. String Programs (31–45)

> 🎯 String manipulation is in every real Python program. These cover the full toolkit.

31 — Reverse a String

```
s = "Hello World"
print(s[::-1])  # dlroW olleH
```

32 — Check Palindrome String

```
s = "racecar"
print("Palindrome" if s == s[::-1] else "Not Palindrome")
```

33 — Count Vowels

```
s = "Hello World"
vowels = sum(1 for c in s.lower() if c in "aeiou")
print(f"Vowels: {vowels}")  # 3
```

34 — Count Words in a String

```
s = "Python is a great language"
print(f"Words: {len(s.split())}")  # 5
```

35 — Check Anagram

```
def is_anagram(s1, s2):
    return sorted(s1.lower()) == sorted(s2.lower())
print(is_anagram("listen", "silent"))  # True
```

36 — Title Case a String

```
s = "the quick brown fox"
print(s.title())  # The Quick Brown Fox
```

37 — Remove Duplicates from String

```
s = "programming"
unique = "".join(dict.fromkeys(s))
```

```
print(unique)  # progamin
```

38 — Most Frequent Character

```
s = "programming"
char = max(set(s), key=s.count)
print(f"Most frequent: '{char}' ({s.count(char)} times)")
```

39 — Caesar Cipher

```
def caesar(text, shift):
    result = ""
    for c in text:
        if c.isalpha():
            base = ord('A') if c.isupper() else ord('a')
            result += chr((ord(c) - base + shift) % 26 + ba
se)
        else:
            result += c
    return result
print(caesar("Hello", 3))  # Khoor
```

40 — Count Occurrences of Substring

```
s = "banana"
print(s.count("an"))  # 2
```

41 — Capitalise First Letter of Each Word

```
words = "hello world python".split()
print(" ".join(w.capitalize() for w in words))
```

42 — Check if String is All Digits

```
for s in ["12345", "123a5", "  "]:
    print(f"{s!r}: {s.isdigit()}")
```

43 — Remove Punctuation

```
import string
s = "Hello, World! How are you?"
clean = s.translate(str.maketrans("", "", string.punctuatio
n))
print(clean)  # Hello World How are you
```

44 — Compress Repeated Characters

```
def compress(s):
    result, i = "", 0
    while i < len(s):
        count = 1
        while i + count < len(s) and s[i] == s[i + count]:
            count += 1
        result += s[i] + (str(count) if count > 1 else "")
        i += count
    return result
print(compress("aaabbbcc"))  # a3b3c2
```

45 — Longest Word in a Sentence

```
sentence = "The quick brown fox jumped"
longest = max(sentence.split(), key=len)
print(f"Longest: '{longest}'")  # jumped
```

🟢 4. List Programs (46–60)

> 🎯 Lists are the most used data structure. These cover every operation you'll need.

46 — Sum and Average of a List

```
nums = [10, 20, 30, 40, 50]
print(f"Sum: {sum(nums)}, Avg: {sum(nums)/len(nums)}")
```

47 — Find Min and Max

```python
nums = [3, 1, 4, 1, 5, 9, 2, 6]
print(f"Min: {min(nums)}, Max: {max(nums)}")
```

48 — Remove Duplicates from List

```python
nums = [1, 2, 2, 3, 4, 4, 5]
unique = list(dict.fromkeys(nums))  # preserves order
print(unique)  # [1, 2, 3, 4, 5]
```

49 — Reverse a List

```python
nums = [1, 2, 3, 4, 5]
print(nums[::-1])         # new reversed list
nums.reverse()             # in-place
print(nums)
```

50 — Sort List Ascending and Descending

```python
nums = [3, 1, 4, 1, 5, 9]
print(sorted(nums))               # ascending
print(sorted(nums, reverse=True)) # descending
```

51 — Flatten a Nested List

```python
nested = [[1, 2], [3, 4], [5, 6]]
flat = [x for sublist in nested for x in sublist]
print(flat)  # [1, 2, 3, 4, 5, 6]
```

52 — List Intersection

```python
a = [1, 2, 3, 4, 5]
b = [3, 4, 5, 6, 7]
print(list(set(a) & set(b)))  # [3, 4, 5]
```

53 — List Union

```python
a = [1, 2, 3]
b = [3, 4, 5]
```

```
print(list(set(a) | set(b)))  # [1, 2, 3, 4, 5]
```

54 — Count Element Occurrences

```
nums = [1, 2, 2, 3, 3, 3, 4]
from collections import Counter
print(Counter(nums))  # Counter({3:3, 2:2, 1:1, 4:1})
```

55 — Rotate a List

```
nums = [1, 2, 3, 4, 5]
n = 2
rotated = nums[n:] + nums[:n]
print(rotated)  # [3, 4, 5, 1, 2]
```

56 — Chunk a List into Groups

```
def chunk(lst, size):
    return [lst[i:i+size] for i in range(0, len(lst), siz
e)]
print(chunk([1,2,3,4,5,6,7], 3))  # [[1,2,3],[4,5,6],[7]]
```

57 — Find Second Largest

```
nums = [10, 20, 4, 45, 99]
sorted_unique = sorted(set(nums), reverse=True)
print(f"Second largest: {sorted_unique[1]}")  # 45
```

58 — Zip Two Lists into Dict

```
keys   = ["name", "age", "city"]
values = ["Alice", 30, "New York"]
d = dict(zip(keys, values))
print(d)
```

59 — Filter Even Numbers

```python
nums = list(range(1, 21))
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)
```

60 — Matrix Addition

```python
A = [[1,2],[3,4]]
B = [[5,6],[7,8]]
C = [[A[i][j] + B[i][j] for j in range(len(A[0]))] for i in range(len(A))]
print(C)  # [[6,8],[10,12]]
```

🔴 5. Dictionaries & Sets (61–75)

> 🎯 Dicts and sets power lookups, frequency counts, and deduplication.

61 — Word Frequency Counter

```python
text = "the cat sat on the mat the cat"
words = text.split()
freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1
print(freq)
```

62 — Merge Two Dictionaries

```python
d1 = {"a": 1, "b": 2}
d2 = {"b": 3, "c": 4}
merged = {**d1, **d2}  # d2 values win on conflict
print(merged)  # {'a':1, 'b':3, 'c':4}
```

63 — Invert a Dictionary

```python
d = {"a": 1, "b": 2, "c": 3}
inverted = {v: k for k, v in d.items()}
```

```
print(inverted)  # {1:'a', 2:'b', 3:'c'}
```

64 — Sort Dict by Value

```
d = {"banana": 3, "apple": 5, "cherry": 1}
sorted_d = dict(sorted(d.items(), key=lambda x: x[1]))
print(sorted_d)  # {'cherry':1, 'banana':3, 'apple':5}
```

65 — Filter Dict by Condition

```
scores = {"Alice": 85, "Bob": 42, "Carol": 91, "Dave": 58}
passed = {k: v for k, v in scores.items() if v >= 60}
print(passed)
```

66 — Nested Dict Access

```
employee = {
    "name": "Alice",
    "address": {"city": "London", "zip": "EC1A"}
}
print(employee["address"]["city"])  # London
print(employee.get("salary", "Not set"))
```

67 — Count Character Frequency

```
from collections import Counter
s = "mississippi"
print(Counter(s).most_common(3))
# [('s',4),('i',4),('p',2)]
```

68 — Set Operations

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}
print(A | B)   # union
print(A & B)   # intersection
print(A - B)   # difference
print(A ^ B)   # symmetric difference
```

69 — Check Subset / Superset

```
A = {1, 2, 3}
B = {1, 2, 3, 4, 5}
print(A.issubset(B))    # True
print(B.issuperset(A))  # True
```

70 — Remove Duplicate Dicts from List

```
records = [{"id": 1, "name": "Alice"}, {"id": 2, "name": "B
ob"}, {"id": 1, "name": "Alice"}]
unique = list({frozenset(d.items()): d for d in records}.va
lues())
print(unique)
```

71 — Default Dict (avoid KeyError)

```
from collections import defaultdict
dd = defaultdict(list)
for k, v in [("a",1),("b",2),("a",3)]:
    dd[k].append(v)
print(dict(dd))  # {'a':[1,3], 'b':[2]}
```

72 — Named Tuple

```
from collections import namedtuple
Point = namedtuple("Point", ["x", "y"])
p = Point(3, 4)
print(p.x, p.y)          # 3 4
print(p._asdict())        # OrderedDict
```

73 — OrderedDict

```
from collections import OrderedDict
od = OrderedDict()
od["first"]  = 1
od["second"] = 2
od["third"]  = 3
```

```python
for k, v in od.items():
    print(k, v)
```

74 — Find Common Keys Between Two Dicts

```python
d1 = {"a": 1, "b": 2, "c": 3}
d2 = {"b": 20, "c": 30, "d": 40}
common = set(d1.keys()) & set(d2.keys())
print(common)  # {'b', 'c'}
```

75 — Frequency Table from List

```python
from collections import Counter
fruits = ["apple","banana","apple","cherry","banana","appl
e"]
table = Counter(fruits)
for fruit, count in table.most_common():
    print(f"{fruit}: {'' * count} ({count})")
```

🩷 6. Control Flow (76–90)

> 🎯 Loops, conditionals, comprehensions — the logic layer of every program.

76 — FizzBuzz (classic)

```python
for i in range(1, 21):
    if   i % 15 == 0: print("FizzBuzz")
    elif i % 3  == 0: print("Fizz")
    elif i % 5  == 0: print("Buzz")
    else:             print(i)
```

77 — Multiplication Table

```python
n = 7
for i in range(1, 11):
    print(f"{n} x {i:2} = {n*i:3}")
```

78 — Sum of First N Natural Numbers

```python
n = 100
print(n * (n + 1) // 2)  # 5050 (Gauss formula)
```

79 — Number Guessing Game

```python
import random
secret = random.randint(1, 100)
while True:
    guess = int(input("Guess (1-100): "))
    if   guess < secret: print("Too low!")
    elif guess > secret: print("Too high!")
    else: print("Correct!"); break
```

80 — Print Triangle Pattern

```python
for i in range(1, 6):
    print("* " * i)
```

81 — Continue and Break Demo

```python
for i in range(10):
    if i == 3: continue  # skip 3
    if i == 7: break     # stop at 7
    print(i, end=" ")    # 0 1 2 4 5 6
```

82 — While Loop with Else

```python
n = 5
while n > 0:
    print(n)
    n -= 1
else:
    print("Loop complete")
```

83 — List Comprehension with Condition

```python
nums = [1, -2, 3, -4, 5, -6]
pos = [x for x in nums if x > 0]
```

```
neg = [x for x in nums if x < 0]
print(f"Positive: {pos}, Negative: {neg}")
```

84 — Nested List Comprehension

```
matrix = [[i * j for j in range(1,4)] for i in range(1,4)]
for row in matrix:
    print(row)
```

85 — Ternary in Comprehension

```
nums = range(1, 11)
labels = ["even" if n % 2 == 0 else "odd" for n in nums]
print(labels)
```

86 — Match Statement (Python 3.10+)

```
command = "quit"
match command:
    case "start": print("Starting...")
    case "stop":  print("Stopping...")
    case "quit":  print("Quitting...")
    case _:       print("Unknown command")
```

87 — For-Else (search with flag)

```
nums = [1, 3, 5, 7, 9]
target = 6
for n in nums:
    if n == target:
        print("Found!")
        break
else:
    print("Not found")  # runs if loop completes without br
eak
```

88 — Enumerate and Zip Together

```
names   = ["Alice", "Bob", "Carol"]
scores  = [85, 92, 78]
for i, (name, score) in enumerate(zip(names, scores), 1):
    print(f"{i}. {name}: {score}")
```

89 — Walrus Operator (Python 3.8+)

```
import random
while (n := random.randint(1, 10)) != 5:
    print(f"Got {n}, not 5")
print("Got 5!")
```

90 — Generator Expression vs List

```
# List: stores all values in memory
squares_list = [x**2 for x in range(1000000)]

# Generator: computes on demand
squares_gen = (x**2 for x in range(1000000))
print(next(squares_gen))  # 0
print(next(squares_gen))  # 1
```

⬛ 7. Functions (91–105)

> 🎯 Functions make code reusable and readable. These cover every function pattern.

91 — Default Arguments

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"
print(greet("Alice"))           # Hello, Alice!
print(greet("Bob", "Hi"))       # Hi, Bob!
```

92 — Args and Kwargs

```python
def summarise(*args, **kwargs):
    print(f"Args: {args}")
    print(f"Kwargs: {kwargs}")
summarise(1, 2, 3, name="Alice", age=30)
```

93 — Return Multiple Values

```python
def stats(nums):
    return min(nums), max(nums), sum(nums)/len(nums)
lo, hi, avg = stats([10, 20, 30, 40, 50])
print(f"Min:{lo} Max:{hi} Avg:{avg}")
```

94 — Lambda Functions

```python
square   = lambda x: x ** 2
add      = lambda x, y: x + y
is_even  = lambda x: x % 2 == 0
print(square(5), add(3,4), is_even(8))
```

95 — Map, Filter, Reduce

```python
from functools import reduce
nums = [1, 2, 3, 4, 5]
print(list(map(lambda x: x**2, nums)))     # squares
print(list(filter(lambda x: x>2, nums)))   # filter >2
print(reduce(lambda x,y: x+y, nums))       # 15
```

96 — Closure

```python
def multiplier(n):
    def multiply(x):
        return x * n
    return multiply
double = multiplier(2)
triple = multiplier(3)
print(double(5), triple(5))  # 10 15
```

97 — Decorator

```
def timer(func):
    import time
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time()-start:.4
f}s")
        return result
    return wrapper

@timer
def slow_sum(n):
    return sum(range(n))
slow_sum(1000000)
```

98 — Memoisation with functools.cache

```
from functools import cache

@cache
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
print(fib(50))  # instant
```

99 — Generator Function

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1
for x in countdown(5):
    print(x, end=" ")  # 5 4 3 2 1
```

100 — Recursive Binary Search

```
def binary_search(arr, target, lo=0, hi=None):
    hi = hi if hi is not None else len(arr) - 1
```

```
    if lo > hi: return -1
    mid = (lo + hi) // 2
    if arr[mid] == target: return mid
    if arr[mid] < target: return binary_search(arr, target,
mid+1, hi)
    return binary_search(arr, target, lo, mid-1)

print(binary_search([1,3,5,7,9,11,13], 7))  # 3
```

101 — Bubble Sort

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
print(bubble_sort([64, 34, 25, 12, 22, 11, 90]))
```

102 — Merge Sort

```
def merge_sort(arr):
    if len(arr) <= 1: return arr
    mid = len(arr) // 2
    left  = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]: result.append(left[i]);  i
+= 1
        else:                   result.append(right[j]); j
+= 1
    return result + left[i:] + right[j:]
print(merge_sort([38, 27, 43, 3, 9, 82, 10]))
```

103 — Stack using List

```
stack = []
stack.append(1)  # push
stack.append(2)
stack.append(3)
print(stack.pop())  # 3 (LIFO)
print(stack)        # [1, 2]
```

104 — Queue using deque

```
from collections import deque
queue = deque()
queue.append("A")    # enqueue
queue.append("B")
queue.append("C")
print(queue.popleft())  # A (FIFO)
print(queue)            # deque(['B', 'C'])
```

105 — Linked List (simple)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self): self.head = None
    def append(self, data):
        node = Node(data)
        if not self.head: self.head = node; return
        cur = self.head
        while cur.next: cur = cur.next
        cur.next = node
    def display(self):
        vals, cur = [], self.head
        while cur: vals.append(cur.data); cur = cur.next
        print(" -> ".join(map(str, vals)))

ll = LinkedList()
```

```
for v in [1,2,3,4,5]: ll.append(v)
ll.display()  # 1 -> 2 -> 3 -> 4 -> 5
```

🔵 8. File Handling (106–115)

> 🎯 Every real program reads and writes files. These cover every file scenario.

106 — Write to a File

```
with open("output.txt", "w") as f:
    f.write("Line 1\n")
    f.write("Line 2\n")
print("Written successfully")
```

107 — Read Entire File

```
with open("output.txt", "r") as f:
    content = f.read()
print(content)
```

108 — Read Line by Line

```
with open("output.txt", "r") as f:
    for line in f:
        print(line.strip())
```

109 — Append to a File

```
with open("output.txt", "a") as f:
    f.write("Line 3\n")
```

110 — Count Lines in a File

```
with open("output.txt") as f:
    lines = f.readlines()
print(f"Lines: {len(lines)}")
```

111 — Read CSV File

```
import csv
with open("data.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row)
```

112 — Write CSV File

```
import csv
rows = [{"name":"Alice","score":95},{"name":"Bob","score":8
7}]
with open("scores.csv","w",newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["name","score"])
    writer.writeheader()
    writer.writerows(rows)
```

113 — Read and Write JSON

```
import json
data = {"name":"Alice","scores":[95,87,91]}
with open("data.json","w") as f:
    json.dump(data, f, indent=2)
with open("data.json") as f:
    loaded = json.load(f)
print(loaded)
```

114 — Check if File Exists

```
import os
if os.path.exists("output.txt"):
    print("File exists")
    print(f"Size: {os.path.getsize('output.txt')} bytes")
else:
    print("File not found")
```

115 — List All Files in a Directory

```python
import os
for f in os.listdir("."):
    if os.path.isfile(f):
        print(f)
```

🟣 9. OOP — Classes (116–125)

> 🎯 Object-oriented programming fundamentals — the patterns used in every large Python project.

116 — Basic Class

```python
class Dog:
    def __init__(self, name, breed):
        self.name  = name
        self.breed = breed
    def bark(self):
        return f"{self.name} says: Woof!"

d = Dog("Buddy", "Labrador")
print(d.bark())
```

117 — Class with Class Variable

```python
class Counter:
    count = 0  # shared by all instances
    def __init__(self):
        Counter.count += 1
    @classmethod
    def get_count(cls):
        return cls.count

a = Counter(); b = Counter(); c = Counter()
print(Counter.get_count())  # 3
```

118 — Inheritance

```
class Animal:
    def __init__(self, name): self.name = name
    def speak(self): return f"{self.name} makes a sound"

class Cat(Animal):
    def speak(self): return f"{self.name} says: Meow!"

class Dog(Animal):
    def speak(self): return f"{self.name} says: Woof!"

for animal in [Cat("Whiskers"), Dog("Rex")]:
    print(animal.speak())
```

119 — Dunder Methods

```
class Vector:
    def __init__(self, x, y): self.x = x; self.y = y
    def __add__(self, other): return Vector(self.x+other.x,
self.y+other.y)
    def __repr__(self): return f"Vector({self.x}, {self.
y})"
    def __len__(self): return int((self.x**2 + self.y**2)**
0.5)

v1, v2 = Vector(1,2), Vector(3,4)
print(v1 + v2)  # Vector(4, 6)
print(len(v1))  # 2
```

120 — Property Decorator

```
class Circle:
    def __init__(self, radius):
        self._radius = radius
    @property
    def radius(self): return self._radius
    @radius.setter
    def radius(self, v):
        if v < 0: raise ValueError("Radius cannot be negati
```

```
ve")
        self._radius = v
    @property
    def area(self): return 3.14159 * self._radius ** 2

c = Circle(5)
print(c.area)   # 78.53
c.radius = 10
print(c.area)   # 314.15
```

121 — Static Method

```
class MathUtils:
    @staticmethod
    def is_even(n): return n % 2 == 0
    @staticmethod
    def clamp(val, lo, hi): return max(lo, min(val, hi))

print(MathUtils.is_even(4))       # True
print(MathUtils.clamp(15, 0, 10)) # 10
```

122 — Dataclass

```
from dataclasses import dataclass, field

@dataclass
class Employee:
    name:   str
    salary: float
    skills: list = field(default_factory=list)

e = Employee("Alice", 75000, ["Python", "SQL"])
print(e)  # Employee(name='Alice', salary=75000, skills=['P
ython','SQL'])
```

123 — Abstract Base Class

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self): pass
    @abstractmethod
    def perimeter(self): pass

class Rectangle(Shape):
    def __init__(self, w, h): self.w = w; self.h = h
    def area(self): return self.w * self.h
    def perimeter(self): return 2 * (self.w + self.h)

r = Rectangle(4, 6)
print(r.area(), r.perimeter())  # 24 20
```

124 — Context Manager (with statement)

```
class Timer:
    import time
    def __enter__(self):
        import time; self.start = time.time(); return self
    def __exit__(self, *args):
        import time; self.elapsed = time.time() - self.star
t
        print(f"Elapsed: {self.elapsed:.4f}s")

with Timer():
    total = sum(range(1000000))
```

125 — Singleton Pattern

```
class Singleton:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

```
a = Singleton()
b = Singleton()
print(a is b)  # True — same object
```

🟠 10. Useful Utility Programs (126–150)

> 🎯 Programs that solve real problems. Copy these directly into projects.

126 — Password Generator

```
import random, string
def gen_password(length=12):
    chars = string.ascii_letters + string.digits + string.p
unctuation
    return "".join(random.choices(chars, k=length))
print(gen_password(16))
```

127 — Email Validator

```
import re
def is_valid_email(email):
    pattern = r"^[\w.-]+@[\w.-]+\.\w{2,}$"
    return bool(re.match(pattern, email))
for e in ["alice@example.com", "not-an-email", "user@.co
m"]:
    print(f"{e}: {is_valid_email(e)}")
```

128 — URL Validator

```
import re
def is_valid_url(url):
    pattern = r"https?://[\w.-]+(?:\.[\w.-]+)+[\w._~:/?#\
[\]@!$&'()*+,;=-]*"
    return bool(re.match(pattern, url))
print(is_valid_url("https://www.example.com"))  # True
```

129 — Flatten Deeply Nested List

```
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list): result.extend(flatten(it
em))
        else: result.append(item)
    return result
print(flatten([1,[2,[3,[4,5]]],6]))  # [1,2,3,4,5,6]
```

130 — Roman Numeral Converter

```
def to_roman(n):
    vals = [(1000,'M'),(900,'CM'),(500,'D'),(400,'CD'),(10
0,'C'),(90,'XC'),
            (50,'L'),(40,'XL'),(10,'X'),(9,'IX'),(5,'V'),
(4,'IV'),(1,'I')]
    result = ""
    for v, sym in vals:
        while n >= v: result += sym; n -= v
    return result
print(to_roman(2024))  # MMXXIV
```

131 — Binary to Decimal and Back

```
n = 42
binary = bin(n)[2:]     # '101010'
octal  = oct(n)[2:]     # '52'
hex_   = hex(n)[2:]     # '2a'
print(binary, octal, hex_)
print(int(binary, 2))   # back to 42
```

132 — Stopwatch / Timer

```
import time
input("Press Enter to start...")
start = time.time()
input("Press Enter to stop...")
```

```
elapsed = time.time() - start
print(f"Time: {elapsed:.2f} seconds")
```

133 — Matrix Transpose

```
matrix = [[1,2,3],[4,5,6],[7,8,9]]
transposed = [list(row) for row in zip(*matrix)]
for row in transposed:
    print(row)
```

134 — Acronym Generator

```
phrase = "As Soon As Possible"
acronym = "".join(w[0].upper() for w in phrase.split())
print(acronym)  # ASAP
```

135 — Text-Based Calculator

```
def calculate(expr):
    try: return eval(expr)
    except: return "Invalid expression"
print(calculate("2 + 3 * 4"))  # 14
print(calculate("100 / 4"))    # 25.0
```

136 — Coin Change Problem

```
def coin_change(amount, coins):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for x in range(coin, amount + 1):
            dp[x] = min(dp[x], dp[x - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
print(coin_change(11, [1, 5, 6, 9]))  # 2
```

137 — Anagram Groups

```
from collections import defaultdict
words = ["eat","tea","tan","ate","nat","bat"]
groups = defaultdict(list)
for w in words:
    groups[tuple(sorted(w))].append(w)
print(list(groups.values()))
```

138 — Run-Length Encoding

```
def rle_encode(s):
    result, i = [], 0
    while i < len(s):
        count = 1
        while i + count < len(s) and s[i] == s[i+count]: co
unt += 1
        result.append((s[i], count)); i += count
    return result
print(rle_encode("AAABBBCCDDDDEE"))  # [('A',3),('B',3),
('C',2),('D',4),('E',2)]
```

139 — Find All Permutations

```
from itertools import permutations
result = list(permutations("ABC"))
print(["".join(p) for p in result])
# ['ABC','ACB','BAC','BCA','CAB','CBA']
```

140 — Two Sum Problem

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        complement = target - n
        if complement in seen:
            return [seen[complement], i]
        seen[n] = i
```

```
    return []
print(two_sum([2, 7, 11, 15], 9))  # [0, 1]
```

141 — Palindrome Linked List Check

```
def is_palindrome(lst):
    return lst == lst[::-1]
print(is_palindrome([1,2,3,2,1]))  # True
print(is_palindrome([1,2,3,4,5]))  # False
```

142 — Frequency Sort

```
from collections import Counter
nums = [1,1,1,2,2,3]
print(sorted(nums, key=lambda x: -Counter(nums)[x]))
# [1,1,1,2,2,3]
```

143 — Rolling Average

```
def rolling_avg(data, window):
    return [sum(data[i:i+window])/window for i in range(len
(data)-window+1)]
print(rolling_avg([1,2,3,4,5,6,7], 3))  # [2.0, 3.0, 4.0,
5.0, 6.0]
```

144 — Retry Decorator

```
import time, random
def retry(times=3, delay=1):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try: return func(*args, **kwargs)
                except Exception as e:
                    print(f"Attempt {attempt+1} failed:
{e}")
                    time.sleep(delay)
            raise Exception("All retries exhausted")
```

```
        return wrapper
    return decorator

@retry(times=3, delay=0.5)
def flaky():
    if random.random() < 0.7: raise ValueError("Random fail
ure")
    return "Success!"
```

145 — Simple Logging Decorator

```
from datetime import datetime
def log(func):
    def wrapper(*args, **kwargs):
        print(f"[{datetime.now():%H:%M:%S}] Calling {func._
_name__}")
        result = func(*args, **kwargs)
        print(f"[{datetime.now():%H:%M:%S}] {func.__name__}
returned {result}")
        return result
    return wrapper

@log
def add(a, b): return a + b
add(3, 4)
```

146 — Batch Process a List

```
def batch(items, size, process_fn):
    for i in range(0, len(items), size):
        batch_items = items[i:i+size]
        process_fn(batch_items)

batch(list(range(1,21)), 5, lambda b: print(f"Processing:
{b}"))
```

147 — Simple Config Reader

```
import json, os
class Config:
    def __init__(self, path):
        with open(path) as f: self._data = json.load(f)
    def get(self, key, default=None):
        return self._data.get(key, default)
# config = Config("config.json")
# print(config.get("db_host", "localhost"))
```

148 — Memoisation Without Library

```
def memoize(func):
    cache = {}
    def wrapper(*args):
        if args not in cache:
            cache[args] = func(*args)
        return cache[args]
    return wrapper

@memoize
def expensive(n):
    print(f"Computing {n}...")
    return n * n
print(expensive(5))  # Computing 5... 25
print(expensive(5))  # 25 (from cache)
```

149 — Progress Bar

```
import time
def progress_bar(total, width=40):
    for i in range(total + 1):
        done  = int(width * i / total)
        bar   = "" * done + "-" * (width - done)
        print(f"\r[{bar}] {i}/{total}", end="", flush=True)
        time.sleep(0.05)
    print()
progress_bar(20)
```

150 — Mini Unit Test Runner

```
def run_tests(test_cases, func):
    passed = failed = 0
    for inp, expected in test_cases:
        result = func(inp)
        status = "✅" if result == expected else "❌"
        print(f"{status} Input: {inp} | Expected: {expecte
d} | Got: {result}")
        if result == expected: passed += 1
        else: failed += 1
    print(f"\nResults: {passed} passed, {failed} failed")

def double(x): return x * 2
run_tests([(1,2),(3,6),(5,10),(0,0),(-1,-2)], double)
```

⬛ Quick Reference — Programs by Concept

Concept	Programs
Variables & Types	01–06
Arithmetic & Math	07–15, 16–30
Strings	31–45
Lists	46–60
Dicts & Sets	61–75
Loops & Control Flow	76–90
Functions & Lambdas	91–95
Closures & Decorators	96–97, 144–145
Recursion	17, 100, 129
Sorting Algorithms	101–102
Data Structures	103–105
File Handling	106–115
OOP & Classes	116–125
Regex	127–128
Algorithms	136–140

Concept	Programs
Utilities	126–150

✅ You're Ready When...

- You can write programs 01–30 from memory without referencing
- You use list comprehensions instead of for loops for transformations
- You use `Counter`, `defaultdict`, and `deque` from collections naturally
- You write functions with `*args`, `**kwargs`, and default arguments
- You understand closures and have written at least one decorator
- You can implement binary search and bubble sort from scratch
- You read and write CSV and JSON files without looking up syntax
- You define classes with `__init__`, `__repr__`, and at least one property
- You use `@dataclass` for simple data holder classes
- You can solve Two Sum, Anagram Groups, and Coin Change from memory